

Digital Systems Design – Assignment 1

Harry Leddy – 23372611

Contents

1.0: Introduction:	1
2.0: Design Strategy:	2
2.1: Reused Modules:	2
2.2: New Modules:	3
3.0: Sub-Module Development:	3
3.1: Ripple Adder:	3
3.2: Bitwise XNOR:	3
3.3: Comparator Development:	4
3.4: Multiplexer Development:	6
4.0: ALU Integration & Testing:	6
4.1: Testing Pass-Through Values:	6
4.2: Negating Operators:	7
4.3: Less-Than Operator:	7
4.4: XNOR Operator:	8
4.5: Addition/Subtraction Operator:	8
5.0: FPGA Implementation:	9
5.1: RTL Design:	9
5.2: I/O Pin Mapping:	9
5.3: Implemented Design:	10
6.0: Demo:	11
Appendix – Lab E:	12
1.0: Code Integration:	12
2.0: Implementation Flow:	15
3.0: Resource Consumption:	18
Appendix – Hand Calculations for Comparator:	19

1.0: Introduction:

An Arithmetic Logic Unit (ALU) is used to perform arithmetic and logical operations on binary numbers. In modern digital systems, ALUs form a fundamental component of processors and embedded hardware, allowing operations such as addition, subtraction and comparison to be performed on input data.

The aim of this assignment was to design, implement, and test a 6-bit mini-ALU using Verilog and deploy it on a Basys 3 FPGA board. The ALU operates on two 6-bit inputs, A and B, represented in two's complement format, and produces a 6-bit output X. A 3-bit FXN determines which operation is performed.

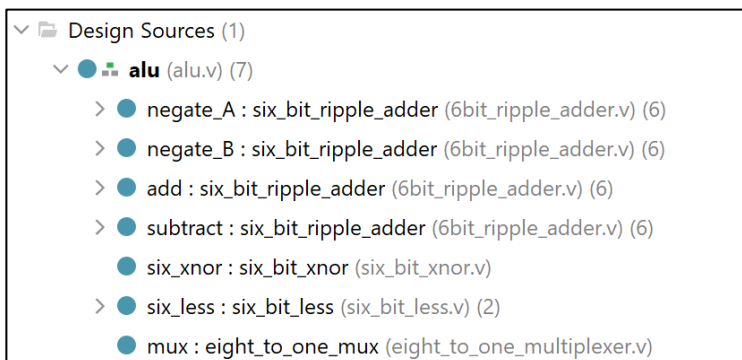
The final design was verified through behavioural simulation in Vivado and demonstrated on the Basys-3 FPGA board using switches as inputs and LEDs as outputs.

2.0: Design Strategy:

The ALU was designed using a modular approach. The overall design was split into several sub-modules, each implementing an operation outlined in the assignment brief. This made the design easier to test incrementally and also ensured that the previously developed components (from the labs) could be reused where possible, as suggested in the brief.

At the top level, the ALU module was implemented as a structural wrapper only. It does not contain behavioural selection logic, it only acts as an orchestrator for all of the sub-modules. It instantiates the submodules for each required operation and routes their outputs into a multiplexer, which selects the final ALU output based on the FXN input. This design structure can be seen in the module hierarchy in

Figure 2. The local file structure on Windows Explorer is also included in Figure 1, as requested



in the brief

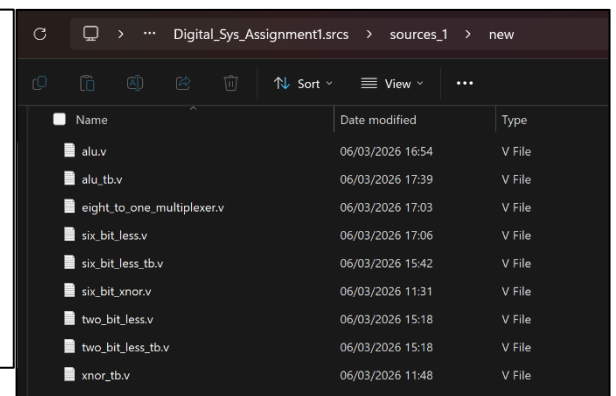


Figure 2 - ALU Modular Design Hierarchy

Figure 1 - Windows File Layout of Source Files

2.1: Reused Modules:

The main reused module was the previously implemented “*six_bit_ripple_adder*”, which had already been designed and tested in the previous lab report. Reusing this module reduced design time significantly.

This ripple adder was reused in four places within the ALU:

- **add:** implements $A + B$
- **subtract:** implements $A - B$
- **negate_A:** implements $-A$ by evaluating $0 - A$
- **negate_B:** implements $-B$ by evaluating $0 - B$

The “*fullAdder*” module was also reused indirectly, it appears within the six-bit ripple adder.

2.2: New Modules:

The “*six_bit_xnor*” module was written to implement the bitwise XNOR operation required in the brief. This performs a bitwise comparison of A and B, producing a 1 wherever the two corresponding bits are equal.

The “*six_bit_less*” module was written to implement the signed comparison $A < B$. This was a more complex implementation than the XNOR block, as the ALU inputs are in 6-bit two’s complement format. A 2-bit less-than or equal-to comparator was first developed and tested, then extended into a 6-bit signed comparator by handling the sign bit separately and chaining the lower-bit comparison stages together.

The “*eight_to_one_mux*” module was written to select which operation result appears at the final output X. This allowed the ALU to remain fully structural as no switch case had to be implemented in the top-level module.

3.0: Sub-Module Development:

3.1: Ripple Adder:

The ripple adder had already been validated in the previous lab submission, so it was unnecessary to re-validate it’s functionality. As previously mentioned, this module was re-used in order to implement addition, subtraction and negating of both variables.

3.2: Bitwise XNOR:

The next module designed was the bitwise XNOR operator. This module compared the bits of the two input vectors and outputs a 1 wherever the corresponding bits are equal.

The operation performed by this module is:

$$X = A \oplus B$$

As seen in Figure 3, this was an extremely simple module implementation as Verilog operators can already perform bit-wise gate logic on vectors.

This module was then tested independently using a simple testbench before being implemented into the wider ALU design.

```

1  module six_bit_xnor(
2      input  [5:0] A, B,
3      output [5:0] Y
4  );
5
6      // performs XNOR on every bit for A and B
7      assign Y = A ~^ B;
8
9  endmodule

```

Figure 3 - XNOR Code Implementation

As seen in Table 1, several test vectors were used in the testbench to ensure the module correctly handled all cases.

Test	A	B	A XNOR B
1	000000	000000	111111
2	111111	111111	111111
3	101010	101010	111111
4	101010	010101	000000
5	110011	100011	101111

Table 1 - XNOR Test Vectors

```

1  `timescale 1ns / 1ps
2
3  module six_bit_xnor_tb;
4
5      reg [5:0] A;
6      reg [5:0] B;
7      wire [5:0] Y;
8
9      six_bit_xnor uut (.A(A), .B(B), .Y(Y));
10
11     initial begin
12
13         A = 6'b000000; B = 6'b000000; #10;
14         A = 6'b111111; B = 6'b111111; #10;
15         A = 6'b101010; B = 6'b101010; #10;
16         A = 6'b101010; B = 6'b010101; #10;
17         A = 6'b110011; B = 6'b100011; #10;
18
19     $finish;
20
21     end
22
23 endmodule

```

As seen in Figure 4, this testbench included vectors which had:

- All bits matching
- No bits matching
- Only some bits matching

Figure 4 - XNOR Testbench Code

The graphical output of this testbench's behavioural simulation can be seen in Figure 5. It clearly shows the sub-module behaving as expected for all test vectors, ensuring that the module was successfully implemented.

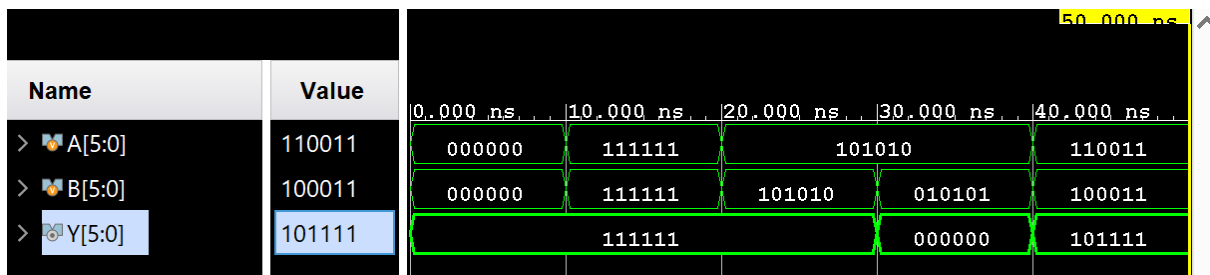


Figure 5 - Graphical Output of XNOR Testbench

3.3: Comparator Development:

In order to implement the 6-bit less-than operator, a 2-bit less-than module was first created.

This was designed by first constructing a truth table describing the desired behaviour of a 2-bit comparison. From this truth table, a Boolean logic expression was derived and implemented using simple logic operators in Verilog. These hand-calculations and truth table were put in the appendix of this report as to not take up unnecessary space, they can be seen in Figure 32.

$$(\overline{A1} B1) + (A1 \oplus B1) \cdot (\overline{A0} B0) \Leftrightarrow A < B$$

The resulting 2-bit module was then tested using its own testbench. All possible input combinations were tested to ensure the correctness of the logic, as shown in Figure 6.

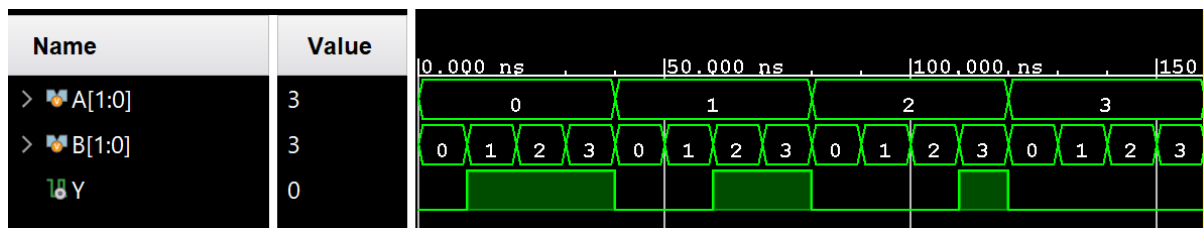


Figure 6 - Graphical Output of 2-Bit Less-Than Testbench

An equality output was also then added to the 2-bit comparator so that multiple comparator modules could be combined into a larger comparison module (a 6-bit less-than comparator). This was then tested using the same testbench, which used all possible combinations of test vectors to ensure correct implementation, this can be seen in Figure 7.

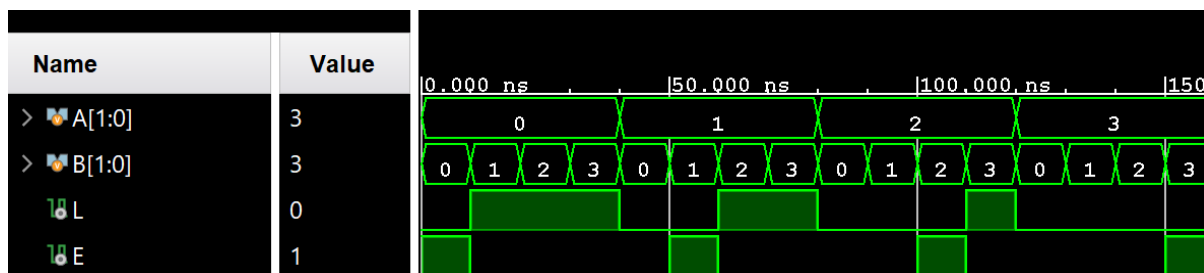


Figure 7 - Graphical Output of 2-Bit Less-Than or Equal-To Testbench

The next step was to construct the full 6-bit less-than comparator by combining the smaller comparator modules. The initial design failed to take the signed two's complement representation of numbers into account. As a result, the first version of the comparator effectively behaved as an unsigned comparator.

To correct this, the most significant bit, which indicated the sign (+/-) of the number, had to be handled separately. The comparator therefore first checks the sign bits of the inputs. The following bit was also treated separately, while the remaining four bits were compared by calling the 2-bit comparator module twice. This structure allowed the comparator to handle signed inputs, the final implementation, with explanatory comments can be seen in Figure 8.

```

1 `timescale 1ns / 1ps
2
3 module six_bit_less(
4     input [5:0] A, B,
5     output [5:0] Y
6 );
7
8     wire top_less; // top bit (after sign indication in MSB)
9     wire top_eq;
10    wire mid_less; // middle two bits
11    wire mid_eq;
12    wire bot_less; // bottom two bits
13    wire bot_eq;
14    wire less_bit; // single bit to store bool result
15
16    // compare the middle two values and see if they are less/equal
17    two_bit_less mid (.A(A[3:2]), .B(B[3:2]), .less(mid_less), .eq(mid_eq));
18    // compare the bottom two values and see if they are less/equal
19    two_bit_less bot (.A(A[1:0]), .B(B[1:0]), .less(bot_less), .eq(bot_eq));
20
21    assign top_less = ~A[4] & B[4];
22    assign top_eq = A[4] ^ B[4];
23
24    assign less_bit = (A[5] & ~B[5]) | // if A is neg and B is not, A must be less
25    ((A[5] ^ B[5]) & // if the signs are the same (both pos or both neg), compare
26    (top_less | // if MSB is less, must be less
27    (top_eq & mid_less) | // if MSB is equal and mid is less, must be less
28    (top_eq & mid_eq & bot_less))); // if MSB and middle two are equal and the bot
29
30    assign Y = {5'b00000, less_bit};
31 endmodule
32

```

Figure 8 - Code for 6-Bit Less-Than Comparator

The final 6-bit comparator was then tested using a testbench. While not every possible input combination was tested, the test vectors included a sufficient range of values to verify correct operation. These included values smaller than A, equal to A, and larger than A, as well as zero and negative values. The graphical results for this testbench can be seen in Figure 9.

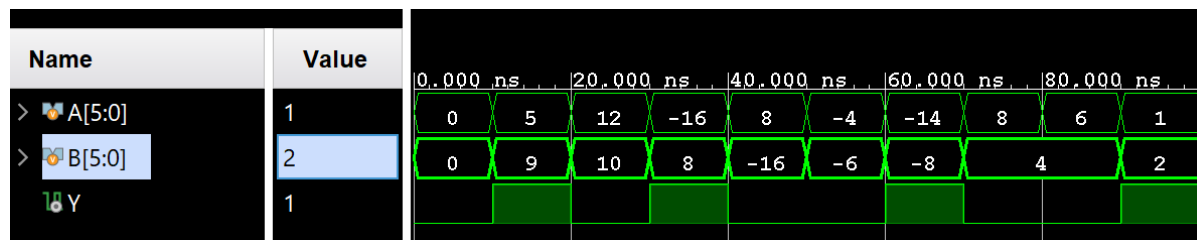


Figure 9 – Graphical Output of 6-Bit Less-Than Comparator Testbench

3.4: Multiplexer Development:

In order to avoid implementing logical selection inside the top-level ALU module, a multiplexer module was created to select which operation result should appear at the output. The multiplexer compares the 3-bit selection input (FXN) and selects one of the eight possible operation outputs accordingly.

This multiplexer was implemented using a switch-case statement, where the selected output is determined by the value of the selection vector.

The multiplexer implementation can be seen in Figure 10.

```

1  `timescale 1ns / 1ps
2
3  module eight_to_one_mux(
4      input [2:0] select,
5      input [5:0] 00, 01, 02, 03, 04, 05, 06, 07, //
6      output reg [5:0] Y
7  );
8
9      always @(*) begin
10         case(select)
11             3'b000: Y = 00;
12             3'b001: Y = 01;
13             3'b010: Y = 02;
14             3'b011: Y = 03;
15             3'b100: Y = 04;
16             3'b101: Y = 05;
17             3'b110: Y = 06;
18             3'b111: Y = 07;
19         endcase
20     end
21
22 endmodule

```

Figure 10 - Code for 8-to-1 Multiplexer

Once all sub-modules had been developed and verified independently, they were integrated into the top-level ALU module and tested as a complete system.

4.0: ALU Integration & Testing:

Once the individual sub-modules had been developed and verified, they were integrated into the top-level ALU module. The multiplexer module was used to switch between the outputs of the different operators to route to the final output X. This allowed the ALU to remain purely structural, with the top-level module acting only as an orchestrator for the previously discussed modules.

After integration, a testbench was written to verify the behaviour of the fully integrated ALU. As required by the assignment brief, the first test vector set input A equal to the binary representation of the board number **18**, which is: **010010**.

4.1: Testing Pass-Through Values:

The initial test cases verified the first two ALU operations, which simply output the values of A and B. As seen in Figure 11, the ALU successfully displayed the correct output for each case depending on the value of the control input FXN.

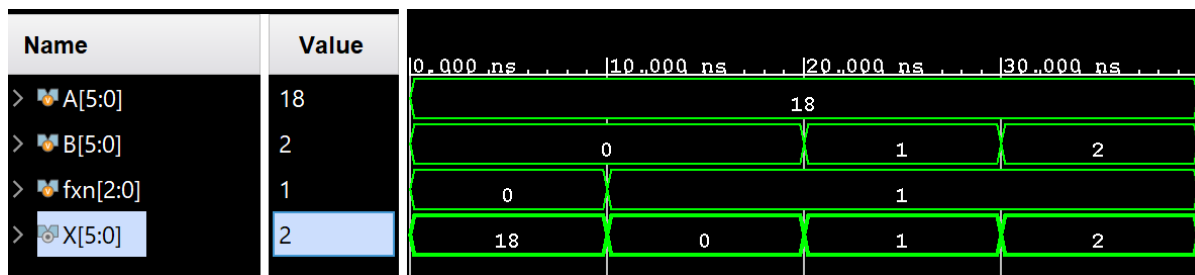


Figure 11 - Graphical Output of ALU Testbench - Passthrough Values

4.2: Negating Operators:

The next operations tested were the negation operations. The first test vector confirmed that the value 18 was correctly negated to -18 using two's complement representation. A range of additional values were then applied to the second input vector (B), and the ALU successfully negated them as expected, as is shown in Figure 12.

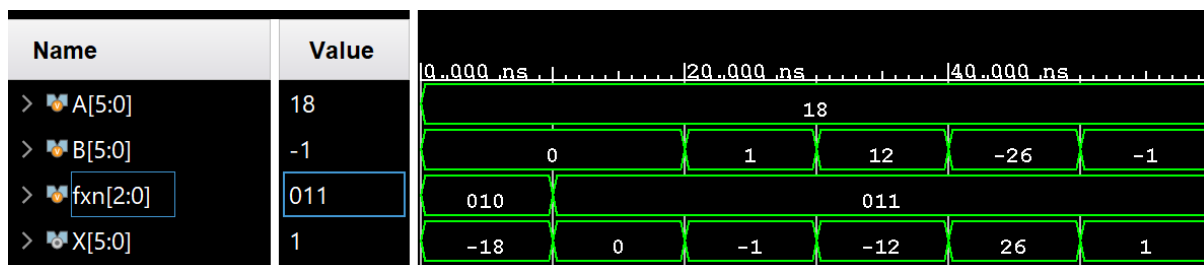


Figure 12 - Graphical Output of ALU Testbench - Negation Operator

4.3: Less-Than Operator:

The $A < B$ operation was then verified. During testing, test vector A remained fixed at 18 while B was varied. Test vectors included values smaller than A, equal to A, and larger than A, as well as zero and negative values. This confirmed that the signed less-than operator was functioning correctly, as shown in Figure 13.

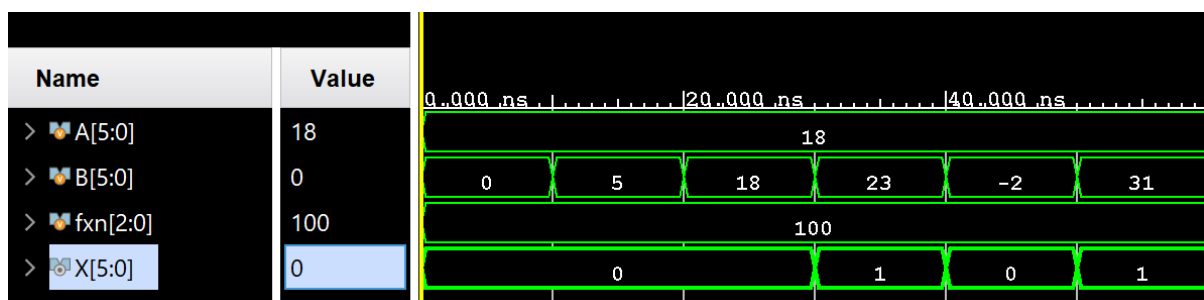


Figure 13 - Graphical Output of ALU Testbench - Less-Than Operator

4.4: XNOR Operator:

The bitwise XNOR operation was tested next. For clarity, the results for this operation were considered in binary form. The XNOR function outputs a one whenever the corresponding bits of A and B are equal, and a zero otherwise. A range of test vectors was therefore selected to include cases where all bits were equal, no bits were equal, and only some bits matched. The simulation results confirmed the expected behaviour of the XNOR operation, as shown in Figure 14.

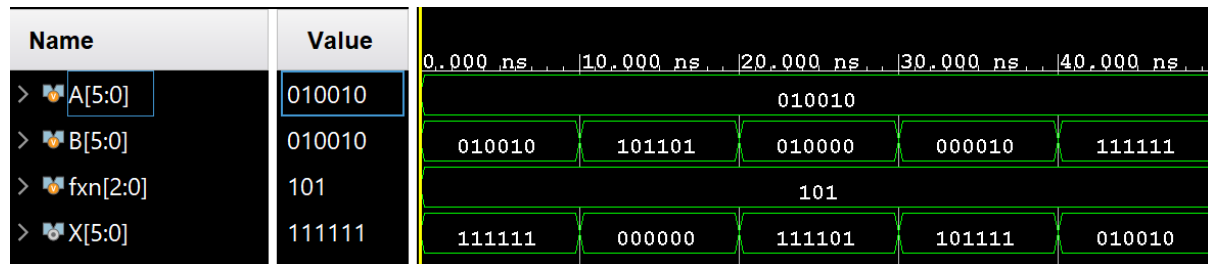


Figure 14 - Graphical Output of ALU Testbench - XNOR Operator

4.5: Addition/Subtraction Operator:

The final two functions tested were addition and subtraction. These operations behaved as expected across the applied test vectors, confirming that the reused ripple adder module had been successfully integrated into the full ALU design. One notable case occurs in the final subtraction test shown in Figure 15, where overflow occurs. In this case, the true expected result is 44, which cannot be represented within the range of a signed 6-bit two's complement number. Although the ripple adder itself generates an overflow flag, this signal is not output by the ALU. As a result, the ALU output wraps according to standard two's complement behaviour.

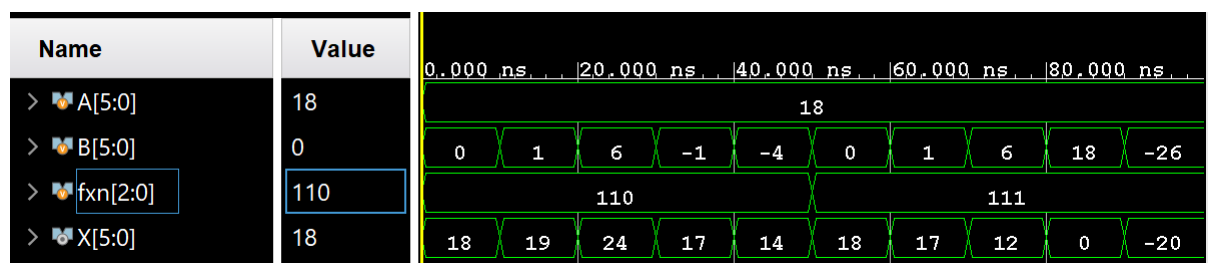


Figure 15 - Graphical Output of ALU Testbench - Add/Subtract Operator

Overall, the integrated ALU testbench confirmed that all eight required operations were functioning correctly and that the sub-modules operated as intended when connected together within the top-level design.

5.0: FPGA Implementation:

5.1: RTL Design:

Once the ALU behaviour was confirmed, through testing, to be functioning as expected, the design was then synthesised. Vivado was used to generate an RTL schematic for the design. This schematic clearly illustrates the hierarchical structure of the ALU, showing the ripple adder modules, comparator, XNOR block and multiplexer connected within the top-level ALU module.

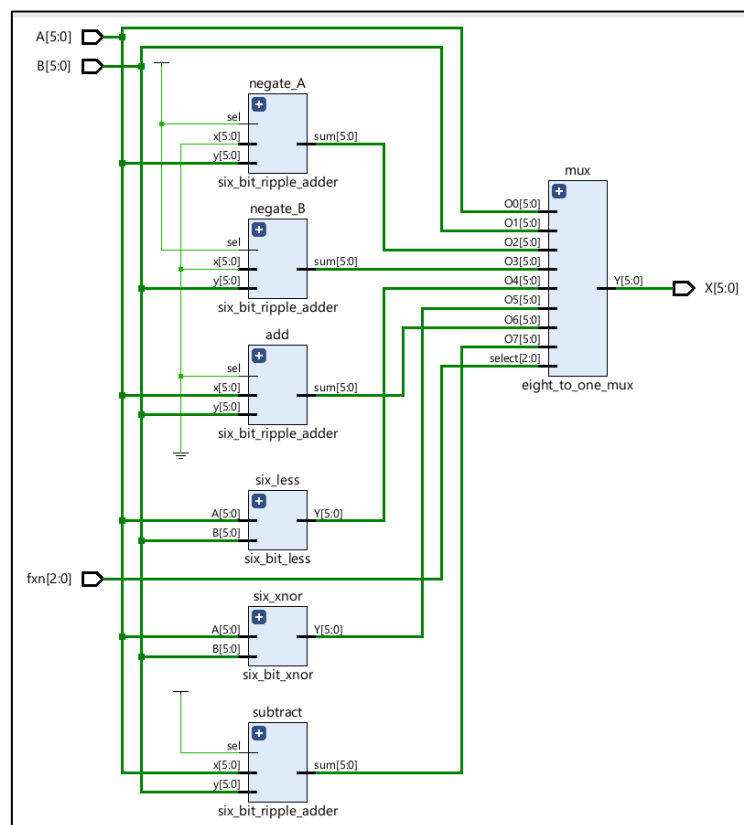


Figure 16 - RTL Schematic of ALU

5.2: I/O Pin Mapping:

As can be seen in Figure 17, Vivado's I/O planner was used to assign the physical FPGA pins to the ALU inputs and outputs. The board's switches were used to provide input values while LEDs displayed the output of the ALU.

For ease of use, the first six LEDs (starting from the right) were assigned to display the 6-bit ALU output X. The first six switches (also starting from the right) were used to represent input B, the next six switches represented input A, and the left-most three switches were used for the FXN input, which selects the ALU operation.



5.3: Implemented Design:

After synthesis and implementation, Vivado generated the implemented design schematic shown in Figure 18. This schematic represents the hardware-level structure produced by the implementation tool and confirms that the design was successfully mapped onto the real-world FPGA resources.

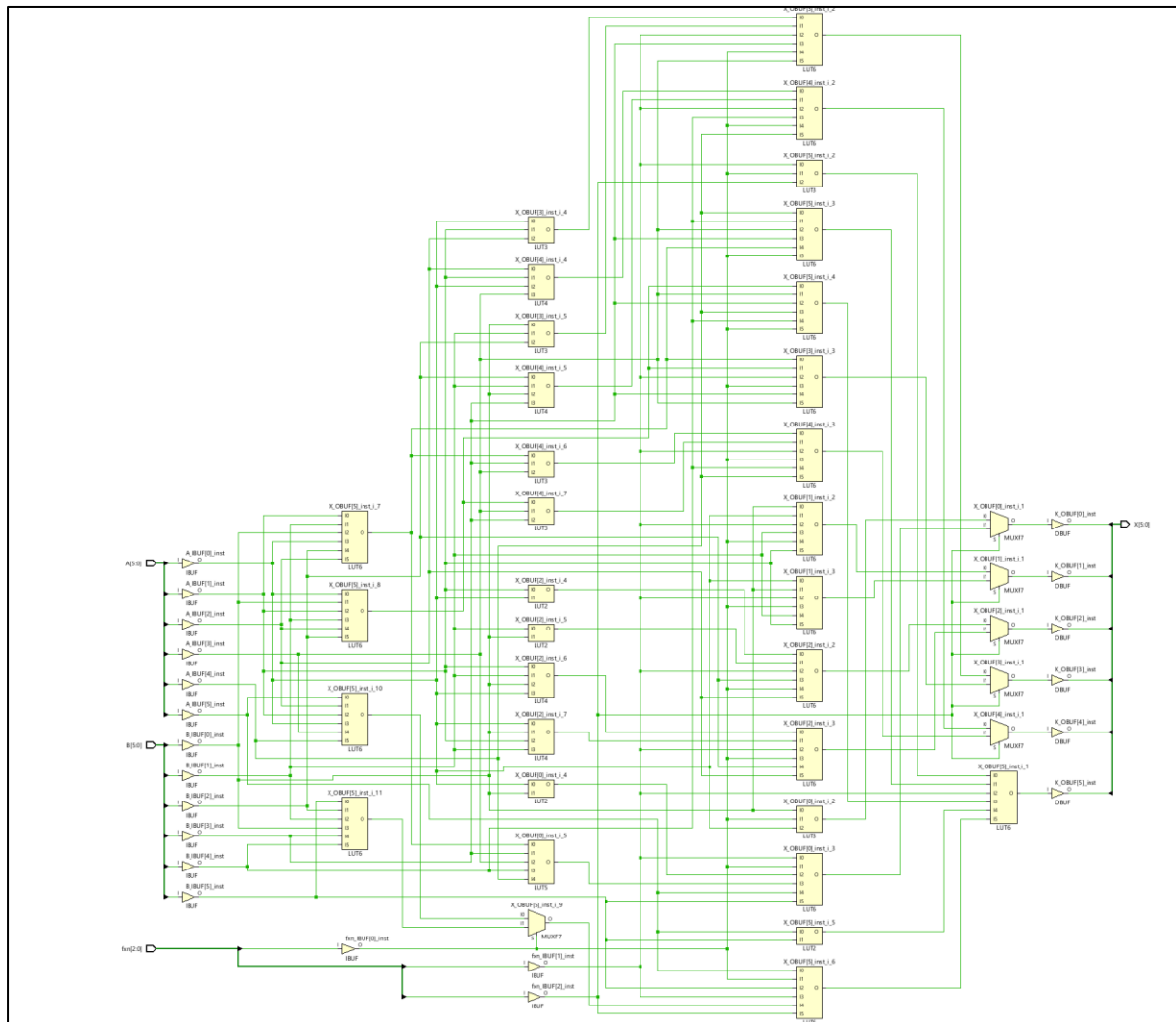


Figure 18 - Implemented Design Schematic

6.0: Demo:

As previously explained, the inputs of the ALU were mapped to the Basys 3 Board's switches while the output was mapped to the LEDs. A labelled diagram can be seen in Figure 20. In this example, the FXN is set to 000, which passes through A to the output. The input A is set to the test vector 010010 which the board number 18. The ALU behaviour is confirmed by the output also showing the binary representation of 18, meaning A was passed through to X.

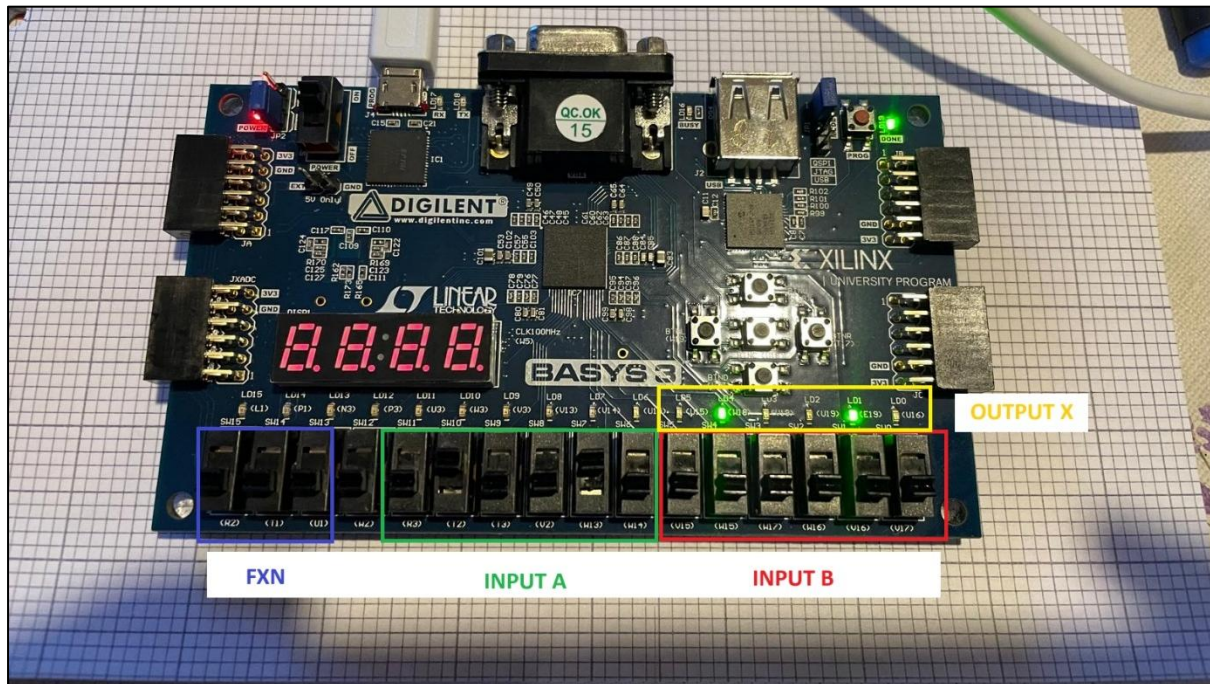


Figure 20 - I/O Board Labelling for Demo

In the example seen in Figure 19, the pass through for B is shown, setting the output to all 1's.

The negating functionality for the test vector A is shown in Figure 21.

The negating functionality for the input vector B was also tested, as seen in Figure 22.

The remaining operations were also tested, but for the sake of brevity, they have not been included in this report as they have already been confirmed through behavioural simulation.

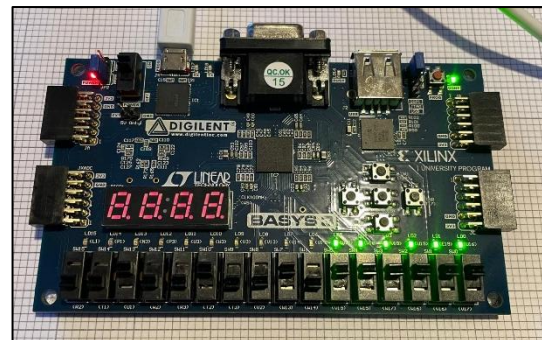


Figure 19 - DEMO - Pass Through B

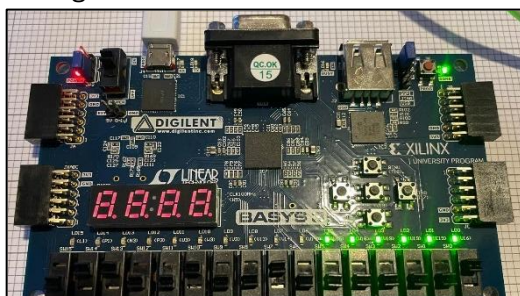


Figure 22 - DEMO - Negate B

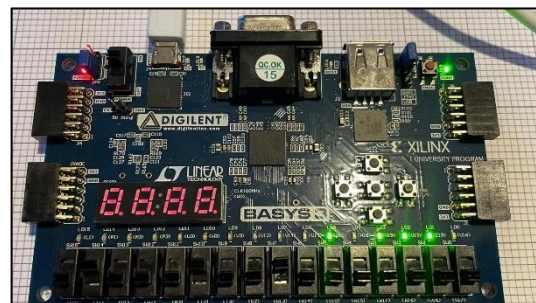


Figure 21 - DEMO - Negate A

Appendix – Lab E:

1.0: Code Integration:

1.1: Full Adder Module:

This module was supplied and provided a starting point for the ripple-adder, this can be seen in Figure 23. This was a simple module which implemented a method to combine two single-bit inputs and a carry. This would then output the addition result along with an output carry.

This was a fundamental building block which could later be chained together to form ripple-adders for chains of any bit length. This could then also be modified to implement functionality such as subtraction and overflow detection.

```

1  module fullAdder(a, b, cin, s, cout);
2      // 3C7 LabD 2010
3      // a and b are the bits to add
4      // cin is carry in
5      input wire a, b, cin;
6
7      // s is the sum of a and b. cout is any carry out bit
8      // wires since just using assign here
9      output wire s, cout;
10
11     // logic for sum and carry
12     assign s = cin ^ a ^ b;
13     assign cout = (b & cin) | (a & cin) | (a & b);
14
15 endmodule

```

Figure 23 - Full Adder Module Code (Given)

The output is calculated by XORing the two input bits along with the carry bit. The carry out bit is calculated by checking if any pair of these bits are both 1.

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = (B \wedge C_{in}) \vee (A \wedge C_{in}) \vee (A \wedge B)$$

1.2: 2-bit Ripple Adder:

The 2-bit ripple adder was created by instantiating two full adders and chaining them together. The carry-out from the least significant bit was connected to the carry-in of the next stage. This implemented the “ripple” effect, where a carry propagates through the chain of full-adders.

The commented code for the 2-bit ripple adder can be seen in Figure 24.

```

1  module two_bit_ripple_adder(
2      input [1:0] x, y,
3      output [1:0] sum,
4      output overflow, c_out
5  );
6
7      // initialise vectors for modified y vector and carry vector
8      wire [2:0] carry;
9      assign carry[0] = 1'b0; // for addition, assume carry in is zero
10
11     // Use full adder module to compute each bit
12     fullAdder add_bit0 (x[0], y[0], carry[0], sum[0], carry[1]);
13     fullAdder add_bit1 (x[1], y[1], carry[1], sum[1], carry[2]);
14
15     // Assign overflow by XORing final 2 carry bits
16     assign overflow = carry[1] ^ carry[2];
17     // Assigning carry out with final carry bit
18     assign c_out = carry[2];
19
20 endmodule

```

Figure 24 - 2-Bit Ripple Adder Code

1.3: 6-bit Ripple Adder:

The 2-bit design was extended to 6 bits by increasing the width of the input, output, and carry vectors and instantiating additional full adder modules. The carry-out of each stage was connected to the carry-in of the subsequent stage, preserving the ripple structure.

```

1  module six_bit_ripple_adder(
2      input [5:0] x, y,
3      output [5:0] sum,
4      output overflow, c_out
5  );
6
7      // initialise vectors for modified y vector and carry vector
8      wire [6:0] carry;
9      assign carry[0] = 1'b0; // for addition, assume carry in is zero
10
11     // Use full adder module to compute each bit
12     fullAdder add_bit0 (x[0], y[0], carry[0], sum[0], carry[1]);
13     fullAdder add_bit1 (x[1], y[1], carry[1], sum[1], carry[2]);
14     fullAdder add_bit2 (x[2], y[2], carry[2], sum[2], carry[3]);
15     fullAdder add_bit3 (x[3], y[3], carry[3], sum[3], carry[4]);
16     fullAdder add_bit4 (x[4], y[4], carry[4], sum[4], carry[5]);
17     fullAdder add_bit5 (x[5], y[5], carry[5], sum[5], carry[6]);
18
19     // Assign overflow by XORing final 2 carry bits
20     assign overflow = carry[5] ^ carry[6];
21     // Assigning carry out with final carry bit
22     assign c_out = carry[6];
23
24 endmodule

```

Figure 25 - 6-Bit Ripple Adder

1.4: Final Design:

The final design implemented addition/subtraction selection to the 6-bit ripple adder. Before writing the code, a block diagram of the design was first created to ensure clear understanding of the structure. This can be seen in Figure 26.

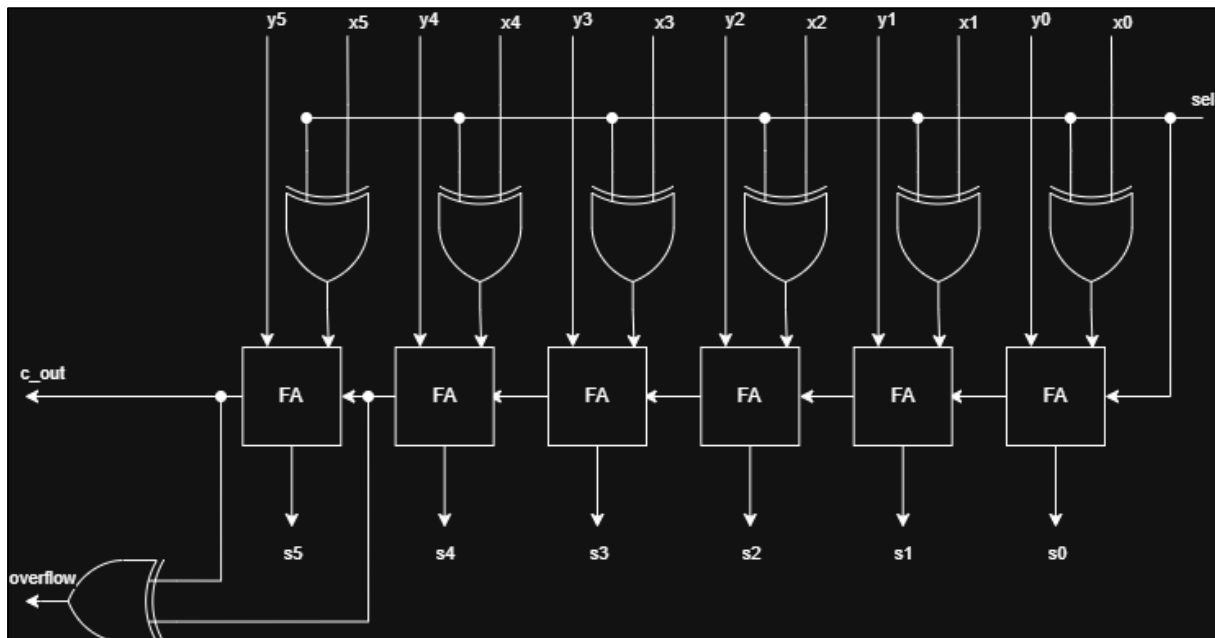


Figure 26 - Block Diagram of Final Design

This setup switches one of the input values to the two's complement negative representation. It flips all the bits (using an exclusive OR) and sets the first carry bit to one. The design also includes an overflow detector by XOR-ing the last two carry bits, as well as a carry-out bit from the final full adder.

Based on this block diagram design, a Verilog module was then written, as seen in Figure 27. The code is commented to explain how it relates back to the original block diagram design. This code was tested using a series of test vectors in Lab D. This compared the expected output to the observed output. The design performed as expected.

```

1  module six_bit_ripple_adder(
2      input [5:0] x, y,
3      input sel,
4      output [5:0] sum,
5      output overflow, c_out
6  );
7
8      // initialise vectors for modified y vector and carry vector
9      wire [6:0] carry;
10     wire [5:0] added_val;
11
12     // set first carry bit to the value of sel
13     assign carry[0] = sel;
14
15     // XOR all y values with sel and store in new vector
16     assign added_val[0] = y[0] ^ sel;
17     assign added_val[1] = y[1] ^ sel;
18     assign added_val[2] = y[2] ^ sel;
19     assign added_val[3] = y[3] ^ sel;
20     assign added_val[4] = y[4] ^ sel;
21     assign added_val[5] = y[5] ^ sel;
22
23     // Use full adder module to compute each bit
24     fullAdder add_bit0 (x[0], added_val[0], carry[0], sum[0], carry[1]);
25     fullAdder add_bit1 (x[1], added_val[1], carry[1], sum[1], carry[2]);
26     fullAdder add_bit2 (x[2], added_val[2], carry[2], sum[2], carry[3]);
27     fullAdder add_bit3 (x[3], added_val[3], carry[3], sum[3], carry[4]);
28     fullAdder add_bit4 (x[4], added_val[4], carry[4], sum[4], carry[5]);
29     fullAdder add_bit5 (x[5], added_val[5], carry[5], sum[5], carry[6]);
30
31     // Assign overflow by XORing final 2 carry bits
32     assign overflow = carry[5] ^ carry[6];
33     // Assigning carry out with final carry bit
34     assign c_out = carry[6];
35
36 endmodule

```

Figure 27 - 6-Bit Ripple Adder with Add/Subtract Selection

2.0: Implementation Flow:

2.1: Synthesis:

Once the Verilog design was complete, it was synthesised using Vivado. During synthesis, the design description was elaborated and optimised, then mapped onto FPGA resources such as LUTs, flip-flops, and dedicated carry logic. The result is a mapped design that can then be passed to implementation.

2.2: I/O Planning & Constraints:

This step involves assigning the logical ports present in the code design to physical pins. In order to do this, Vivado's I/O planner was used. The planner then generates an XDC constraints file, which can be seen in Figure 28.

```

1  set_property IOSTANDARD LVCMOS33 [get_ports {sum[5]}]
2  set_property IOSTANDARD LVCMOS33 [get_ports {sum[4]}]
3  set_property IOSTANDARD LVCMOS33 [get_ports {sum[3]}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {sum[2]}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {sum[1]}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {sum[0]}]
7  set_property IOSTANDARD LVCMOS33 [get_ports {x[5]}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {x[4]}]
9  set_property IOSTANDARD LVCMOS33 [get_ports {x[3]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {x[2]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {x[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {x[0]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {y[5]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {y[4]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {y[3]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {y[2]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {y[1]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {y[0]}]
19 set_property PACKAGE_PIN V17 [get_ports {x[0]}]
20 set_property PACKAGE_PIN V16 [get_ports {x[1]}]
21 set_property PACKAGE_PIN W16 [get_ports {x[2]}]
22 set_property PACKAGE_PIN W17 [get_ports {x[3]}]
23 set_property PACKAGE_PIN W15 [get_ports {x[4]}]
24 set_property PACKAGE_PIN V15 [get_ports {x[5]}]
25 set_property PACKAGE_PIN R2 [get_ports {y[5]}]
26 set_property PACKAGE_PIN T1 [get_ports {y[4]}]
27 set_property PACKAGE_PIN U1 [get_ports {y[3]}]
28 set_property PACKAGE_PIN W2 [get_ports {y[2]}]
29 set_property PACKAGE_PIN R3 [get_ports {y[1]}]
30 set_property PACKAGE_PIN T2 [get_ports {y[0]}]
31 set_property PACKAGE_PIN U16 [get_ports {sum[0]}]
32 set_property PACKAGE_PIN E19 [get_ports {sum[1]}]
33 set_property PACKAGE_PIN U19 [get_ports {sum[2]}]
34 set_property PACKAGE_PIN V19 [get_ports {sum[3]}]
35 set_property PACKAGE_PIN W18 [get_ports {sum[4]}]
36 set_property PACKAGE_PIN U15 [get_ports {sum[5]}]
37
38 set_property PACKAGE_PIN T3 [get_ports sel]
39 set_property IOSTANDARD LVCMOS33 [get_ports sel]
40 set_property PACKAGE_PIN L1 [get_ports c_out]
41 set_property PACKAGE_PIN P1 [get_ports overflow]
42 set_property IOSTANDARD LVCMOS33 [get_ports c_out]
43 set_property IOSTANDARD LVCMOS33 [get_ports overflow]

```

Figure 28 - XDC Constraints File

In this example:

- Each input vector was assigned 6 adjacent switches.
- The “select” port was assigned to a switch in between the input values.
- The output vector was assigned to the final 6 LEDs.
- The carry out and overflow ports were assigned to the first 2 LEDs.

The corresponding pin numbers for each of these components were identified from the Basys 3 board and assigned in the I/O planner, as seen in Figure 29. These defined constraints and pin assignments were then applied during implementation.

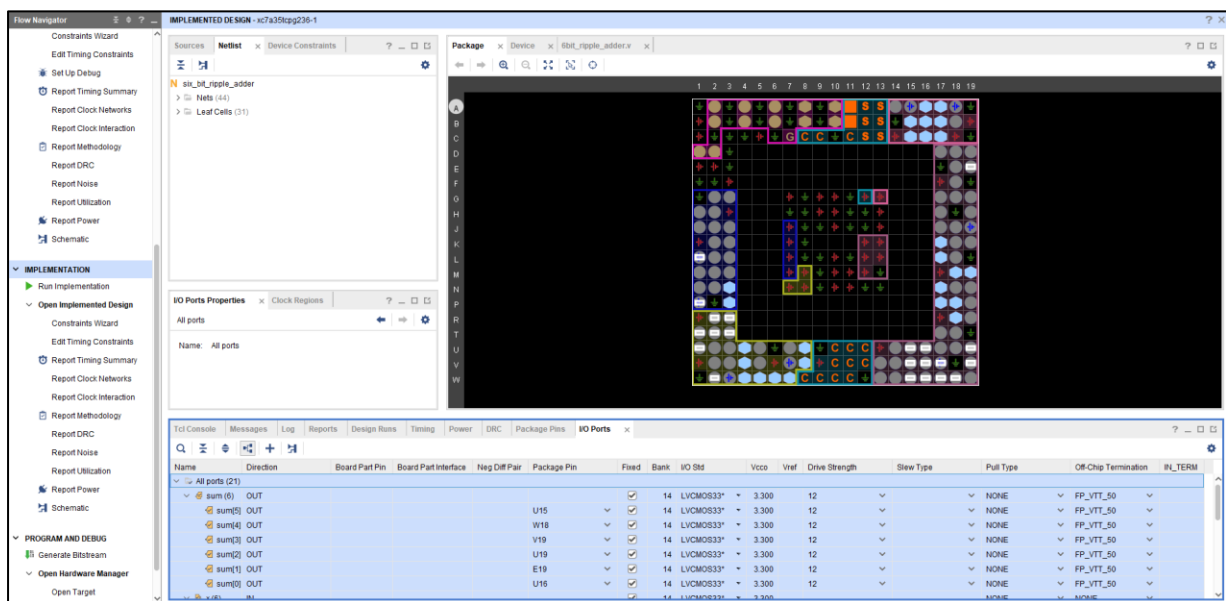


Figure 29 - I/O Planner

2.3: Implementation:

After synthesis and constraint definition, Vivado performed placement and routing. During placement, the mapped logic elements were assigned physical locations on the FPGA board. Routing then connected these elements using the available resources.

The implemented schematic produced by Vivado can be seen in Figure 30.

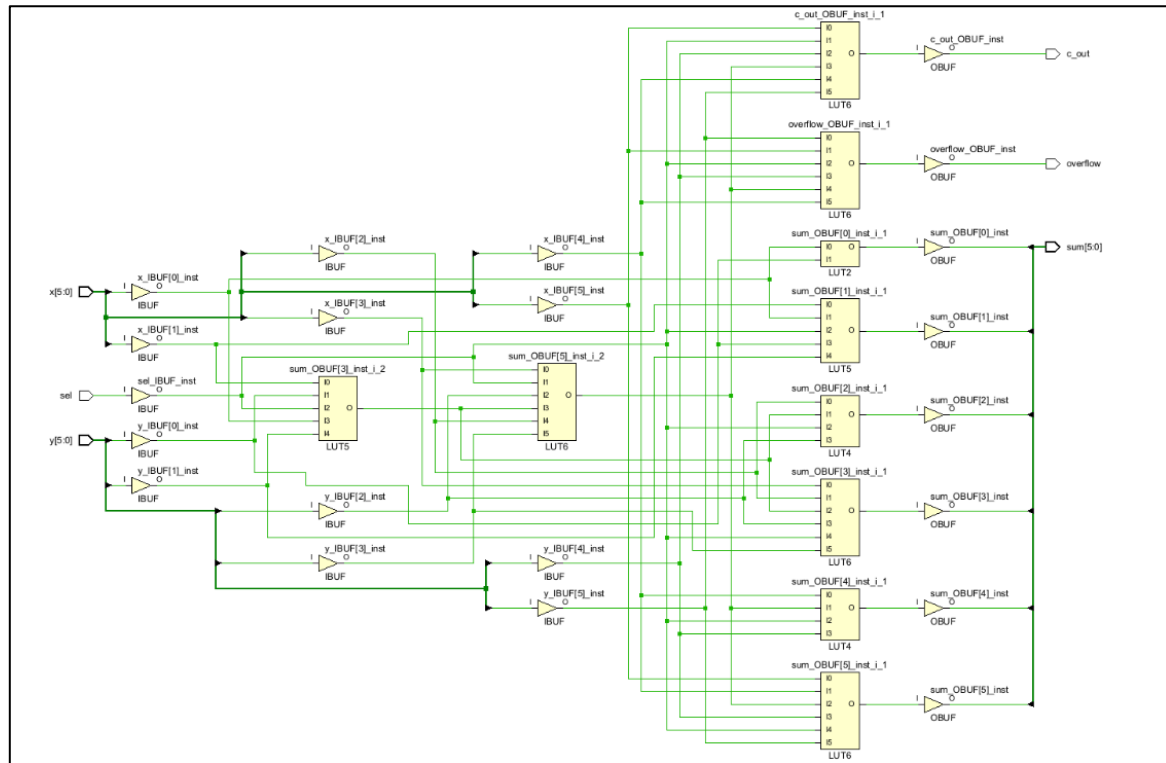


Figure 30 - Implemented Schematic of Design

2.4: Bitstream Generation:

Once implementation completed successfully, a bitstream file was generated and programmed onto the Basys 3 board.

The design was tested using the physical switches as inputs and LEDs as outputs. Test cases consistent with the earlier test vectors were applied as a benchmark (including addition, subtraction, carry-out cases, and signed overflow cases), and the observed LED outputs matched the expected results.

3.0: Resource Consumption:

The Vivado utilization report shows that the ripple adder consumes a small number of FPGA resources. The majority of the logic is implemented using LUTs, which create the XOR and carry logic of the full adder stages. As the ripple adder is a purely combinational circuit, no flip-flops are required. The remaining resources correspond primarily to the I/O pins used to connect the design to the Basys-3 board switches and LEDs.

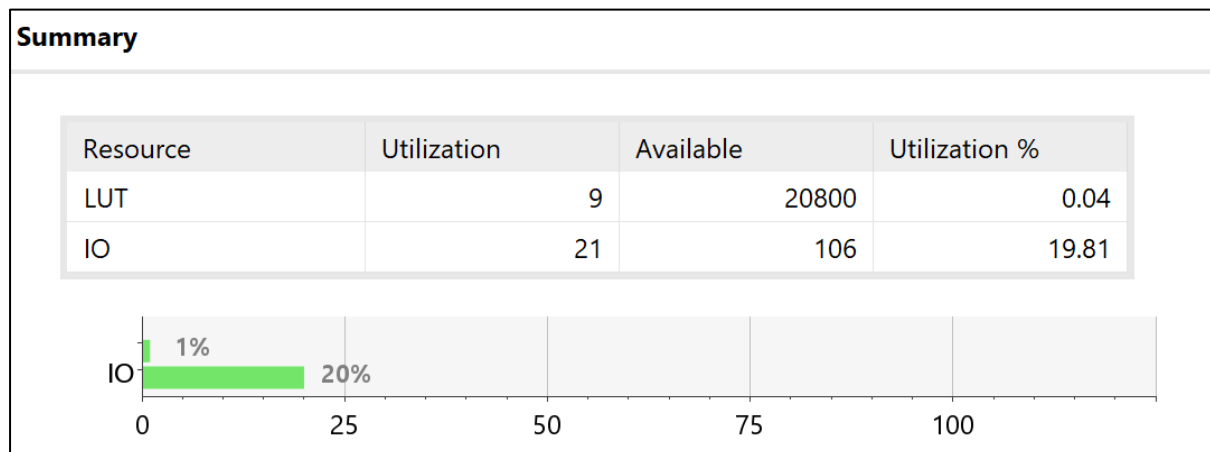


Figure 31 - Ripple-Adder Resource Utilisation

Appendix – Hand Calculations for Comparator:

Handwritten calculations for a 2-bit comparator on graph paper.

Top Bit Analysis:

top	mid	bot
0	1	0
0	0	1

XNOR Truth Table:

0	0	1
0	1	0
1	0	0
1	1	1

Truth Table for Y:

A1	A2	B1	B2	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Logic Derivation:

$A1=0 \& B1=1$ OR $A1 \neq B1 \& B2=1 \& A2=0$

~~$\sim A1 \& B1 \parallel (A1 \& B1) \& (\sim A2 \& B2)$~~

~~$(A1 \& B1) \parallel (\sim A1 \& \sim B1)$~~

~~$(A1 \& B1)$~~

Final Logic:

$(\sim A1 \& B1) \parallel (A1 \neq B1) \& (\sim A2 \& B2)$

For 6 bit is MSB

$A[5]=1 \quad B[5]=0 \quad A < B$

$A[5]=0 \quad B[5]=1 \quad A > B$

$(A \& \sim B) \parallel \dots$

Figure 32 - Hand Calculations for 2-Bit Comparator